ELI — Background Tasks

ELI v2.5.34

November 2026

Contents

ELI Background Tasks & Unified Code Generation **1**
1. Unified code generation 1
2. Background task manager (`eli/runtime/background_tasks.py`) 2
Control summary 2
Scope / honesty 3

Updated for v2.4.84.

ELI Background Tasks & Unified Code Generation

Two related additions: 1. **All code generation now routes through the verified coding agent** — asking for a script, or self-upgrading, runs the plan → DAG/tree-search → execute → repair → bug-memory pipeline instead of a single LLM shot. 2. **Heavy work runs on background threads** — when a task is estimated heavy (or you ask), ELI starts it on a worker thread, hands back a job id immediately, and you check on it later.

1. Unified code generation

Path	Before	Now
`CODE_SOLVE`	the coding agent	the coding agent (unchanged)
`GENERATE_SCRIPT`	inline single-shot prompt + static checks	**routes through `eli.coding.solve` first** (plan/DAG/tree-search/verify/repair), saving a verified, top-tier script; falls back to the inline generator only if the agent yields nothing or `ELI_GENERATE_SCRIPT_AGENT=0`
Self-upgrade (`self_improvement.generate_code_patch`)	LLM patch + import-verify	now also **consults the coding engine's long-term bug memory**: classifies the failure and injects prior fixes for that bug class into the patch prompt

So "write a script", "generate code", and "improve ELI" all share one engine and one bug-memory — the same consolidation move applied to `recall_memory`. Kill switch: `ELI_GENERATE_SCRIPT_AGENT=0` restores the legacy inline generator.

2. Background task manager (`eli/runtime/background_tasks.py`)

In-process, multi-threaded (`ThreadPoolExecutor`). **Distinct** from `eli/planning/jobqueue.py` (which runs durable *external subprocess* jobs) — this runs live in-process Python work (agent runs) for the session.

- `BackgroundTasks.submit(name, fn, *args)` → integer job id; tracks `queued → running → done/failed/cancelled`, result, error, elapsed, note.
- `get(id)`, `list()`, `cancel(id)` (best-effort), `wait(id)`, `stats()`.
- Singleton `get_background_tasks()`; thread-safe.
- **What a job's status means.** A handler that returns `ok: False` failed, whatever the thread did; `skipped` is not a failure. `elapsed` counts from the start, so a job waiting for its time reads 0.
- **The record outlives the session.** Jobs are kept in `data_dir/runtime/background_jobs.json` for 30 days; numbers carry on, and a job that was queued or running when ELI closed loads as `interrupted`: reported, not resumed. `work_lines` and `work_block` read it for job questions and for any message about files, folders, jobs or progress (`eli/runtime/live_reads.py`), so the model answers from the record, and `work_targets` is what "the rest" or "them" refers to.
- **Jobs are the person's.** ELI's own housekeeping (kind `housekeeping`) stays out of `list()`, so "did the job finish?" is never answered about one; the Tasks tab asks for it. Sizes and windows are the `cog.jobs_*` tunables. Asked what a job found, the model reads its report (`latest_report`).
- **When a job ends** (`add_listener`). A failed one goes on the failure record self-analysis reads (`_record_failed_job`); one someone asked for reaches them by desktop notification and the proactive queue (`_tell_the_person`). What goes wrong after a job is kept on the job, `meta.after_job_error`.
- **Model calls from a job.** Daemons get an empty reply while a person is talking (`ELI_BG_DEFER_WINDOW`). A job is started by a turn, so that blanked every job: it now waits, and re-runs a call that a turn cut short (`gguf_inference.set_job_inference`).

When ELI backgrounds a task (`eli/coding/cost.py`)

`should_background(task)` decides: - **Explicit phrasing wins:** "in the background / don't wait / notify me when" → background; "right now / quickly / wait for it" → foreground. - **Otherwise heuristic:** `estimate_cost` scores heavy-compute signals (simulate/monte-carlo/optimise/benchmark/train/dijkstra/parallel…), multi-component structure, length, and planned-step count. `heavy` (≥0.6) ⇒ background.

`_maybe_background_codegen` (executor) applies this at the top of `CODE_SOLVE` and `GENERATE_SCRIPT`: if heavy/explicit, it submits the same action as a background job (re-dispatched with `_no_background` so the worker doesn't re-background) and replies *"started as job #N — say 'check job N'."* Kill switch: `ELI_CODEGEN_BACKGROUND=0`.

Inspecting jobs

- Actions **`CHECK_JOB`** (`{job_id}` or parsed from "check job 3") and **`BACKGROUND_JOBS`** (list), both in `SUPPORTED_ACTIONS`.
- Router triggers: "check job N" / "job N", "background jobs" / "list jobs", and while a job runs a question about its progress or time left ("do you have an eta?", "is it done yet?"), which goes to the running job (`_JOB_PROGRESS`); "how long is the Eiffel Tower" does not.
- **Progress.** A job reports `background_tasks.report(done, total, stage)` from its own thread; `CHECK_JOB` and the Tasks tab show the stage, how far it has got and the time left in that stage at the rate measured so far (`Task.eta_s`). The folder analysis reports reading and summarising. Without this an ETA question got "several hours" for a ten-minute job.

Control summary

Knob	Default	Effect
`ELI_GENERATE_SCRIPT_AGENT`	on	GENERATE_SCRIPT uses the coding agent (off ⇒ legacy inline)
`ELI_CODEGEN_BACKGROUND`	on	auto-background heavy codegen (off ⇒ always foreground)
`ELI_AGENT_DAG` / `ELI_CODING_DAG`	on	DAG dispatch / subtask DAG (see `dag.md`)
`ELI_CODING_SANDBOX` / `*_RUN_TIMEOUT` / `*_BEAM` / `*_MAX_ITERS`	—	coding-agent execution knobs

Scope / honesty

- **Tested deterministically** (`tests/test_background.py`): manager run/fail reporting, cost light-vs-heavy + explicit phrasing, and the CHECK_JOB / BACKGROUND_JOBS executor round-trip. No model needed.
- **Threads, not processes:** background tasks share the process; a running thread can't be force-killed (cancel is best-effort / cooperative). True OS isolation for a job uses the durable subprocess `jobqueue`: run `eli-jobs worker` and queue with `eli-jobs submit --cmd "..."` (jobs survive a restart; one a dead worker left running is marked failed on the next worker start).
- **"LLM deems longer wait":** currently a deterministic estimator (+ explicit phrasing). The planner's decomposition (`plan_steps`) can be fed in for an LLM-informed estimate — hook present, not yet auto-wired.
- **GUI:** the Tasks tab (`eli/gui/tabs/tasks_tab.py`) is the single home for background work: coding jobs and scheduled or overnight tasks (add, edit, cancel). Jobs are also reachable in chat ("check job N", "background jobs") and through the action API.